LogiEdge Cold Chain Monitoring

Group 35 AIML ZG535 Machine Learning on Edge

BITS Pilani Work Integrated Learning Programmes

Instructor Pravin Yashwant Pawar

Group member	Student ID
NANDHINI G	2025AF05070
GANESAN V	2025AE05074
VINAYAK SHARMA	2025AE05018
AHAMED IMTHIAS J	2025AE05985

Executive summary

LogiEdge demonstrates local cold-chain monitoring for the FreightBridge scenario using a Windows laptop, Ubuntu under WSL2 and Docker. A simulated truck publishes sensor readings to Mosquitto. Shared causal preprocessing produces six features for a TFLite classifier, while independent temperature rules provide immediate operational escalation. SQLite retains events and retries deliveries after backend interruption. Three model variants achieved 100% validation accuracy on 59 synthetic windows. M1 was fastest on the measured laptop; M3 was smallest. The live P(Normal) PSI experiment detected distribution shift in 60.00 seconds and recovered after clean restoration. Two Ansible runs demonstrated seven-task deployment and an unchanged second run. These results support a reproducible software pilot. Physical sensors, ARM64 deployment, field validation and measured on-truck power remain necessary before production adoption.

Narrative word count 2553 excluding tables figures and references

1 FreightBridge deployment context

FreightBridge's scenario covers 85 refrigerated trucks, with possible expansion to 265 vehicles. Two-hour manual logs cannot support a 90-second response to refrigeration faults. We therefore place sensing, classification and durable incident storage on the truck. The operations centre receives alerts when communication permits. Our implementation represents one truck on a laptop; fleet costs are projections, and the operations backend is a separate local container [1].

Latency. A 1 degree Celsius per minute temperature rise makes the 90-second requirement operationally important. Connected-network delay alone does not disqualify cloud inference. TRAI's September 2025 Pune assessment reports operator median latencies of 14.1-54.0 ms; these city and surrounding-area measurements do not establish rural tail latency or cloud response guarantees [4]. Even a stated planning allowance of 500 ms network delay plus one second processing fits 90 seconds while connected. Coverage loss, queueing and tail behaviour are the decisive risks. Our controlled combined-fault run observed a local physical Critical alert at 35.08 seconds and a model Critical result at 40.16 seconds.

Bandwidth. Assuming float32 values, temperature requires four bytes per second and 500 Hz three-axis vibration requires 6,000. The numerical payload is 518.7456 decimal MB per truck daily, costing Rs 51.87456 at Rs 0.10/MB. With an explicit assumption of 100 door records daily at 32 bytes each, this becomes 518.7488 MB and Rs 51.87488. A planning example of 1,440 one-KB heartbeat messages plus 20 one-KB alerts uses 1.46 MB and Rs 0.146 daily, a 99.72% numerical-payload reduction. Message rates, encoding and protocol overhead must be measured in production. The simulator's 0.5 Hz vibration RMS is an already summarised stream, distinct from this raw 500 Hz business calculation.

Connectivity. A 35-90 minute outage lasts 2,100-5,400 seconds, approximately 23-60 times the entire alert budget. Local decisions and logging continue through these gaps. Remote receipt necessarily waits for connectivity. Seven queued events survived service recreation in our test and subsequently received acknowledgements. Privacy. Local inference reduces raw-data exposure, but contractual protection also needs authenticated access, encryption, secure updates and audit controls. Our private-network HTTP prototype does not demonstrate those production protections. In deployment, access control must cover the physical node, maintenance interface and backend, not only the radio link. Cargo identifiers and retained logs need explicit retention policies. Hash verification detects accidental artifact changes; it does not replace signed updates or authorisation.

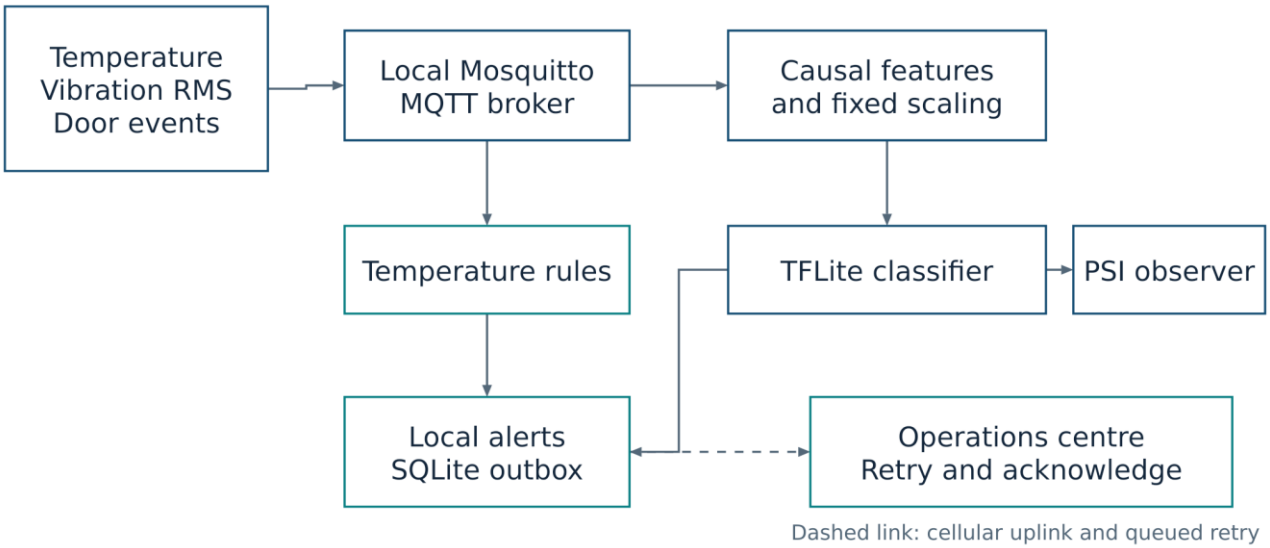


Figure 1 Truck-side intelligence and operations-centre delivery. The dashed path represents the proposed cellular uplink; the prototype uses a local backend.

Module 2 balances Compute Performance, Power Budget and Unit Cost [2]. The 10 W budget makes power the dominant constraint. Using the assignment's indicative figures, Pi 5 with AI HAT+ costs Rs 12.75 lakh for 85 trucks and Rs 39.75 lakh for 265 at 7.5 W. Jetson's 15 W exceeds budget. STM32 is inexpensive but requires a different embedded runtime and orchestration architecture.

Hardware option	Rs per truck	Power W	85 trucks Rs lakh	265 trucks Rs lakh
Pi 5 with AI HAT+	15,000	7.5	12.75	39.75
Jetson Orin Nano Super	45,000	15	38.25	119.25
STM32H7 custom design	3,500	0.4	2.975	9.275

Table 1 Assignment prices and power assumptions. Proposed target selection, not measured hardware consumption.

For the assigned Roofline example, 45 MFLOP divided by 18 MB gives 2.5 FLOP/byte. The ridge is 16 GFLOP/s divided by 12 GB/s, or 1.333 FLOP/byte. Compute dominates, with an ideal 2.8125 ms bound. This hypothetical workload differs from our tiny measured MLP.

Assigned Roofline quantity	Calculated result
Arithmetic intensity	2.5 FLOP/byte
Ridge point	1.333 FLOP/byte
Compute lower bound	2.8125 ms
Memory lower bound	1.5 ms
Classification	Compute bound

2 Sensor pipeline and MQTT design

The simulator generates temperature at 1 Hz around 4 degrees Celsius with standard deviation 0.3, vibration RMS at 0.5 Hz around 0.45 g with standard deviation 0.05, and timestamped door transitions. Temperature drift adds 0.08 degrees per reading. The vibration fault changes its distribution to mean 1.2 g and standard deviation 0.15. Combined mode activates both. Explicit seeds and unique run identifiers make captures reproducible without mixing independent runs. We retain manifests, acknowledged counts and raw JSONL files.

The topic prefix is logibridge/trucks/TRUCK_001. Temperature, vibration_rms and door_event feed inference; inference and alerts carry its local outputs. All use QoS 1 with non-retained messages. QoS 1 permits redelivery, so message identifiers and database uniqueness checks handle duplicates. The consumer acknowledges after processing and committing events. Retaining old incident messages would risk confusing a fresh run with a historical condition. Heartbeats travel through the HTTP outbox to the operations service rather than an additional sensor topic.

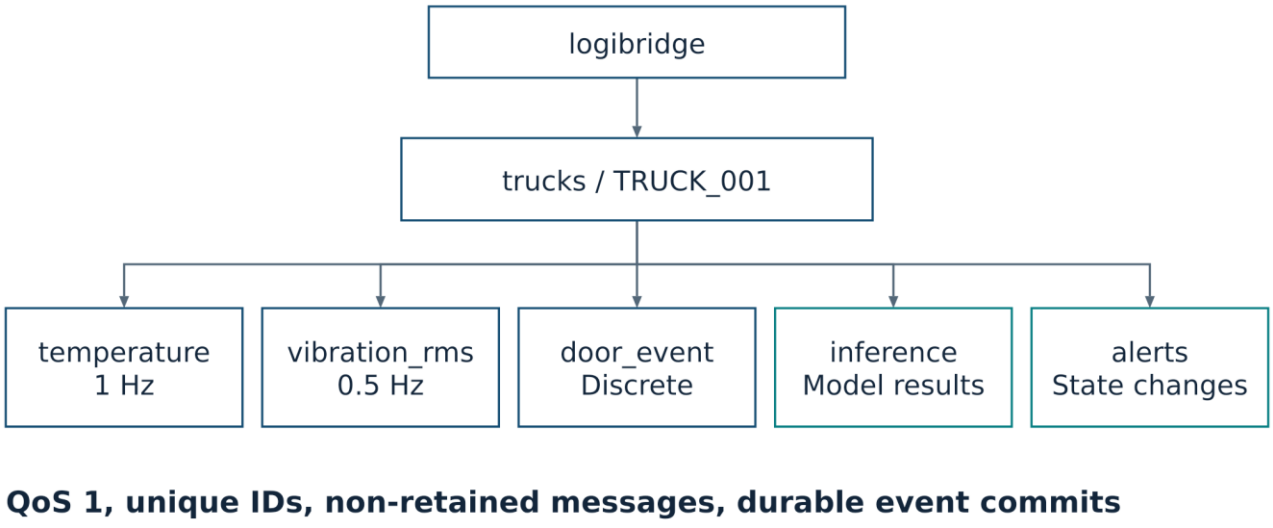


Figure 2 Complete truck MQTT topic tree. All five leaves use QoS 1. Heartbeats use the HTTP outbox. Broker health is logibridge/system/broker_health.

Both numerical streams use five-sample trailing averages. Filtering uses only current and past readings and discards incomplete warm-up. Each full 30-second window contains 30 temperature and 15 vibration readings, and windows advance every ten seconds. Full filtering support makes the first usable window approximately 10-40 seconds after start. Missing or out-of-order readings cause explicit rejection rather than invented replacement values. This collection delay is separate from model invocation latency. Five samples span four seconds of temperature history but eight seconds of vibration history. We therefore align complete filtered windows rather than equating sample counts across streams. A new run resets filter state, preventing the preceding experiment from supplying a fresh experiment's warm-up.

Feature-level fusion concatenates temperature mean, population standard deviation and fitted rate of change with vibration RMS, absolute peak and Fisher excess kurtosis. The slope uses actual sample timestamps and converts seconds to minutes. Constant vibration receives a documented finite kurtosis sentinel. Door state remains contextual information. Labels and anomaly names never enter the numerical vector. Raw data-level concatenation would require unnecessary alignment of asynchronous event and numerical streams. Decision-level fusion would need separate models and rules to reconcile their outputs. Six physically interpretable features provide one compact model interface while preserving sensor-specific preprocessing.

An independent ten-minute clean run with seed 1001 produced 57 complete feature windows. We computed and froze six means and standard deviations in training_stats.npy, with feature-order metadata and a SHA256 identity. Runtime loads and transforms only. It never fits statistics on incoming anomalies. The required three-sigma experiment shifted the saved means by three original standard deviations and retained the original standard deviations. Correct and shifted statistics both yielded 100% M1 validation accuracy, a measured change of zero percentage points. This result does not establish that incorrect preprocessing is generally harmless; the synthetic classes remain strongly separated. Shared feature code and recipe metadata reduce training-serving skew. Tests check duplicates, warm-up, exact feature ordering, closed-window causality and missing support. Dataset construction validates normalisation provenance before training, making an accidental change observable instead of silently producing incompatible inputs.

M1 normalisation experiment	Accuracy on 59 validation windows
Correct statistics	100.00%
Means shifted by three original standard deviations	100.00%
Shifted minus correct	0.00 percentage points

Operational severity combines the latest model class with independent raw-temperature limits around the assignment's 4 degree setpoint. Deviations greater than one degree trigger Warning and greater than three trigger Critical. The model may still be warming up when a physical alarm occurs. SQLite stores local events and an outbox. The backend confirms matching event identifiers before deletion from the pending queue, enabling retries without duplicate logical incidents. The completed post-deployment run acknowledged 71 sensor messages, produced a fresh Normal prediction and left zero pending uploads; local acknowledgements and remote unique events both reached 72.

3 Model deployment and Pipeline Mapping

We map the Module 4 lifecycle to ten explicit activities below. Its slide 4 groups Collect, Label and Augment inside Data; we separate collection from labelling and augmentation to make the ten assessed activities explicit, while preserving the remaining course labels [3].

1 Data collection. We capture independent Normal, temperature-drift and combined runs for 20, 15 and 15 minutes. The resulting 291 feature windows retain capture provenance.

2 Labelling and augmentation. Scenario manifests assign the prescribed classes. Seeded noise supplies variation; we do not claim additional augmentation or relabel using model predictions.

3 Train. TensorFlow trains a CPU-based 6-32-16-3 ReLU MLP with 803 parameters. Training-only conditioning is folded into its first kernel without changing the frozen normaliser.

4 Validate. Chronological per-run holdouts and purging separate overlapping raw support: 223 training, 59 validation and nine discarded windows. Validation data never enter calibration.

5 Optimise. We compare FP32, full INT8 and structured-pruned INT8 variants. PolynomialDecay gates entire hidden units before physical compaction and fine-tuning.

6 Convert. TFLite exports preserve the six-input, three-output interface. Integer audits verify signed INT8 input/output and absence of floating-point fallback.

7 Deploy. Docker packages preprocessing, inference and result publication. Ansible copies the selected model and matching reference, then manages the pinned image.

8 Infer. One CPU thread executes the verified model. Local classification and temperature rules operate independently of backend availability.

9 Monitor. The live observer evaluates output-confidence PSI and records drift, recovery and clock continuity. Saved reference and model identities remain fixed.

10 Update. A proposed canary rollout promotes verified model/reference pairs and retains rollback assets. We demonstrated model-layer distribution; autonomous fleet rollout remains proposed.

The inference Dockerfile installs dependencies before its final model COPY. MODEL_PATH successfully loaded registered M3 inside the existing M1 image without rebuilding. The measured OTA rebuild changed one filesystem layer and reused earlier steps. Registry objects gave a 347,760,955-byte cold target payload and a 13,994-byte cached update, including manifest/configuration. For 85 cached trucks this projects to 1.189490 MB instead of 29,559.681175 MB, saving 99.995976%. These are compressed registry-payload calculations excluding protocol and cellular overhead, not an observed fleet transfer. The cached update comprises a 1,642-byte compressed model layer and 12,352 bytes of new configuration and manifest. Content-addressed identities connect the measurement to the inspected images. Repeating a build after both variants are cached can reuse even the model layer, so the original recorded comparison remains the evidence.

For E3's separate 280 KB model assumption, full replacement sends 23.8 MB and costs Rs 2.38 per 85-truck cycle. Canary first sends 2.8 MB to ten trucks for Rs 0.28; promotion adds Rs 2.10, preserving the Rs 2.38 full-fleet total. Shadow also needs 23.8 MB of new weights if the old model is cached, plus unquantified diagnostic traffic and duplicate computation. We recommend canary to limit fault exposure and support interrupted downloads and rollback.

Full replacement exposes all trucks simultaneously; shadow adds power and analysis overhead.

E3 model-only strategy	Completed 85-truck payload	Completed cost
Full replacement	23.8 MB	Rs 2.38
Canary then promotion	2.8 + 21.0 MB	Rs 0.28 + Rs 2.10
Shadow with previous model cached	23.8 MB	Rs 2.38

Table 2 Separate E3 280 KB assumption. Shadow telemetry, duplicate compute and transport overhead are additional.

4 Optimisation and Pareto analysis

All variants use the same held-out 59 windows: 24 Normal, 18 Warning and 17 Critical. Each achieved 100% accuracy with a diagonal confusion matrix of 24, 18 and 17. Critical recall is 17/17, or 100%, exceeding the specified 95% point threshold. This small synthetic evaluation cannot establish performance across routes, sensors, seasons or real failure mechanisms. Independent field data and substantially more Critical examples are required before safety claims.

M2 uses 223 distinct training-only calibration rows, exceeding the required 200. Its full INT8 file is 3,552 bytes, 34.42% below M1's 5,416. M3 removes 11 of 32 first-layer units and six of 16 second-layer units. Seventeen of 48 units is 35.42%, the whole-unit approximation to the requested 35%. Physical compaction produces a 6-21-10-3 network with 400 parameters. Its INT8 file is 3,216 bytes, 40.62% below M1. Masked-versus-compacted probabilities differed by at most approximately 1.14e-12 before subsequent training.

Both integer models clip 35 validation feature values, affecting 35 rows, while retaining correct class predictions. These saturations are explicitly recorded rather than corrected using validation-derived calibration. The unbounded drift creates values beyond the training calibration range. Future development should use realistic operating envelopes, independent calibration coverage and operationally consistent labels. The prescribed D1 temperature-drift class remains Warning even when its physical temperature exceeds Critical limits; our separate temperature guard makes that mismatch visible.

Benchmarking ran on the team's Intel i5-13420H laptop through Docker/WSL2 using TensorFlow 2.14.1 TFLite built-in CPU kernels, one interpreter thread and no default delegates. Ten warm-up calls preceded 200 measured invocations per variant. Rotation of variant order reduced systematic ordering effects. The five-metric table reports invoke-only latency, not filtering, message transport or storage. We independently recomputed mean and p95 from the saved 600 timing samples. All model hashes and recorded confusion matrices matched.

Variant	Mean ms	p95 ms	KB	Accuracy %	Proxy mJ
M1 FP32	0.001496	0.001684	5.416	100.00	0.005632
M2 INT8	0.002161	0.002342	3.552	100.00	0.008154
M3 structured INT8	0.002109	0.002225	3.216	100.00	0.007986

Table 3 Five measured metrics per variant. Decimal KB. Model-invoke timing. Energy is a proxy.

The energy value is a proxy: process CPU utilisation divided across 12 logical processors scales the 45 W processor-base-power estimate, then multiplies by mean invocation time [5]. A separate five-second CPU probe supplies utilisation. Hybrid cores, simultaneous multithreading, frequency scaling, idle consumption and virtualisation limit this approximation. It is neither measured battery energy nor evidence that the truck node consumes 45 W.

M1 and M3 form the feasible latency-size Pareto frontier. M3 dominates M2 because it is smaller and faster with equal measured accuracy. We select M1 for the verified CPU demonstration because it has the lowest mean, p95 and energy proxy. Reserving approximately 40 seconds for initial collection leaves about 50,000 ms of a 90-second illustrative budget for processing and local alerting. M1's measured p95 is 0.001684 ms. Its 5.416 KB file is small relative to the proposed Pi's 8 GB RAM and microSD storage, but activation/runtime memory and sustained target latency still need profiling. An ARM64-compatible build and target measurements must precede the 85-truck hardware deployment. Model-file bytes alone do not describe peak working memory. Python, TensorFlow, activation tensors and queues also consume RAM. Accelerator TOPS cannot substitute for a supported graph, compatible operators and an actual target benchmark.

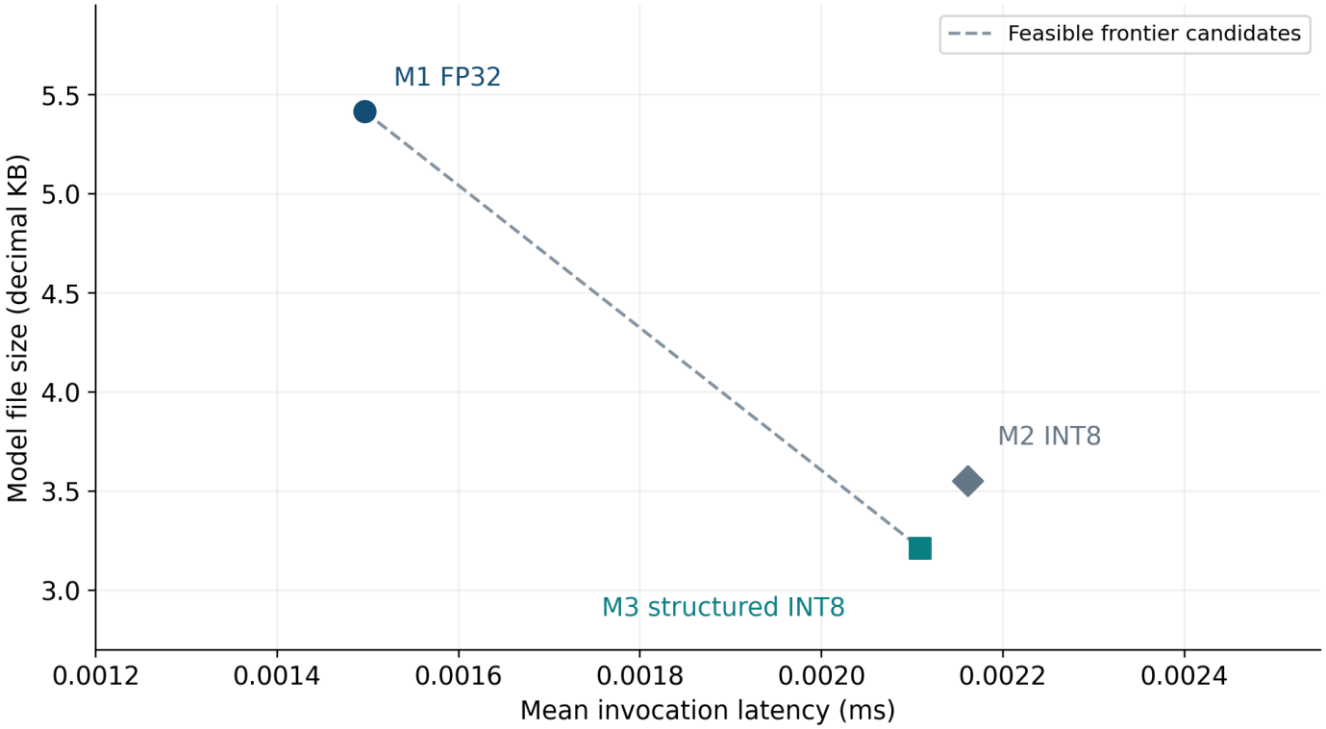


Figure 3 Measured latency-size comparison. M1 and M3 are nondominated feasible candidates; M3 dominates M2. All models have 100% validation accuracy and 17/17 Critical recall.

5 MLOps monitoring and reflection

We define primary confidence as the model's probability of Normal, P(Normal). The frozen reference contains 300 independently generated clean windows. Four fixed bins partition probability space, with exact one included in the final bin. An explicit 1e-6 empty-bin floor and renormalisation keep PSI finite. The live monitor uses the last 100 distinct inference events, evaluates every 60 seconds and retains the same buffer across both mode changes. Maximum-class-confidence PSI is also retained as a diagnostic.

The completed run 2291a861fa424c138da62d062037e160 lasted 2,460.018 monotonic seconds and recorded 242 actual MQTT model outputs. Baseline PSI was zero. After combined injection, the first drift PSI was 0.425487 at 60.000262 seconds, within five minutes. Restoring clean readings produced PSI zero after 1,079.001771 seconds. An independent replay reproduced all 24 full-window evaluations. The reference stayed unchanged, deadlines were met and the clock guard required no interruption review. Maximum-class-confidence PSI stayed zero because both Normal and Critical predictions were confident. We therefore state the class-specific score explicitly rather than claiming maximum confidence detected drift.

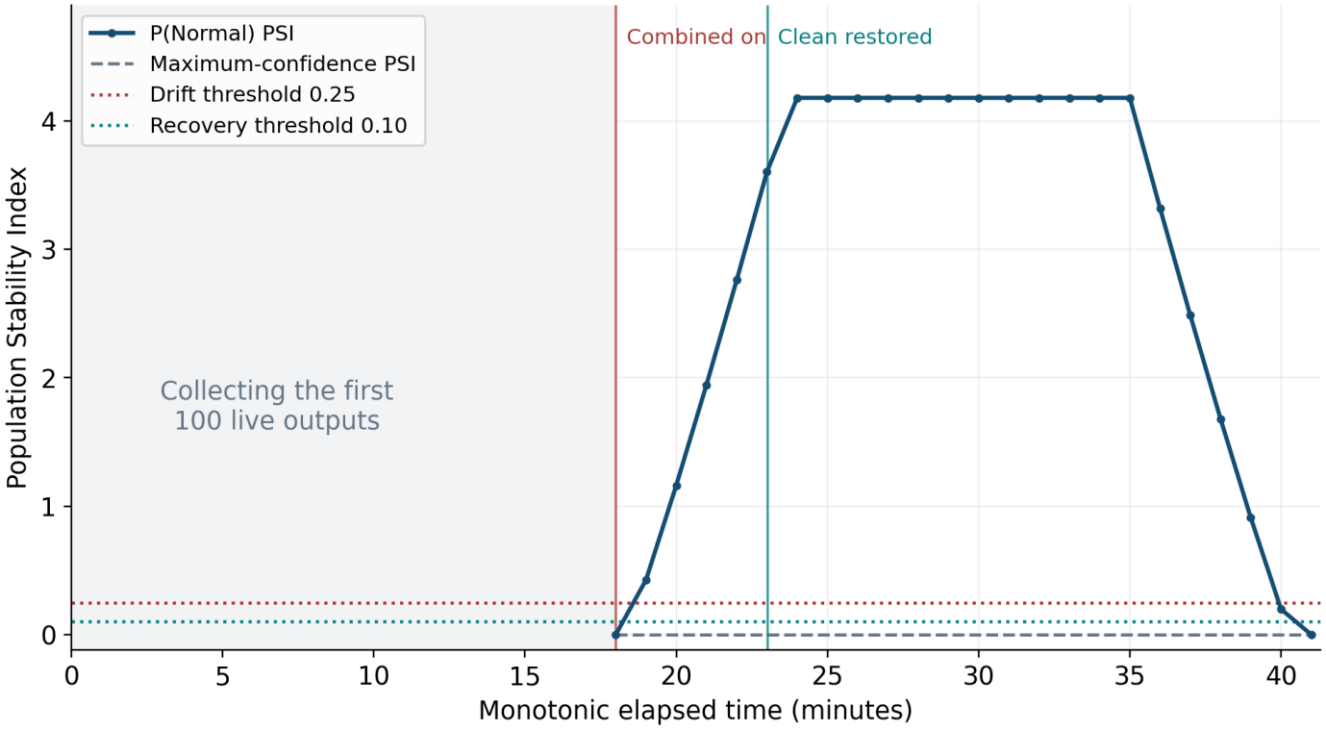


Figure 4 Actual saved live PSI measurements from run 2291a861. Shading marks collection of the first 100 outputs. The rolling history remains intact across injection and restoration.

This injected operating-regime change shifts input and output distributions. It does not establish concept drift or an accuracy decrease, because no real-world target relationship was shown to change. Slow PSI recovery reflects the remembered abnormal outputs, whereas immediate cargo alarms use the separate operational path.

The seven-task Ansible deployment produced ok=7, changed=5 on its first run and ok=7, changed=0 on the unchanged second, with zero failures or unreachable hosts. Identical container IDs and start timestamps independently confirmed no hidden restart. Both verification tasks waited 15 seconds and checked the loaded model, reference and normalisation identities. A fresh 45-second run then verified Normal operation and acknowledged delivery.

A genuine difficulty was distinguishing adjustable UTC from elapsed time under WSL. The original guard rejected an early run. The corrected guard uses monotonic intervals, separately checks processing/suspend gaps and records UTC corrections. Historical UTC-derived alert durations became negative during those corrections and are excluded from latency evidence. One proposed architectural improvement is explicit clock-domain metadata with monotonic latency measurement, alongside authenticated uplink and immutable versioned model/reference deployment. The passing run recorded 28 UTC adjustment notices

and approximately 67.96 seconds cumulative UTC offset. Its largest processing gap was 0.270 seconds. These diagnostics explain why adjustable calendar time must not serve as an elapsed-time stopwatch.

Ansible run	Tasks OK	Changed	Failed
Initial	7	5	0
Unchanged repeat	7	0	0

References and measured evidence

TRAI, Press Release 127/2025, 31 October 2025, Pune September 2025 independent drive test. https://www.trai.gov.in/sites/default/files/2025-10/PR_No.127of2025.pdf

Intel, Core i5-13420H specifications, Processor Base Power 45 W and 12 threads. https://www.intel.com/content/www/us/en/products/sku/232173/intel-core-i513420h-processor-12m-cache-up-to-4-60-ghz/specifications.html

Raspberry Pi, Raspberry Pi 5 specifications and storage options. https://www.raspberrypi.com/products/raspberry-pi-5/

Docker, cache invalidation and Compose operation. https://docs.docker.com/build/cache/invalidation/ and https://docs.docker.com/reference/cli/docker/compose/up/

Python, time clocks. https://docs.python.org/3.11/library/time.html

Ansible, command changes and privilege escalation. https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_error_handling.html

Evidence index

Evidence	Location
Dataset and training provenance	training/dataset/dataset_manifest.json
Selected benchmark	optimisation/results/benchmark_pointer.json
Model-layer OTA measurement	reports/ota_pointer.json
Live PSI audit	reports/psi_live_20261104T044934Z_L9uSpi/audited_summary.json
Two-run deployment	reports/ansible_20261104T062754Z_C85o1J/ansible_results.json
Current exported runtime snapshot	reports/post_deployment_status.json